

Chapter 10: Virtual Memory

Short & Easy Study Note — ICT 3101 Operating System

1. Why Virtual Memory? (Background)

- A full program does not need to be in RAM all at once. Many parts (error-handling code, rare features, big tables) are rarely used.
- If we load only the needed parts, we save I/O, use less RAM per program, and can run more programs at once.
- Result: better CPU utilization and more throughput.

2. What is Virtual Memory?

Simple meaning

Virtual Memory makes a small RAM feel very large. It does this by using part of the disk (secondary memory) as if it were extra RAM.

How it works

- A process can be bigger than the actual RAM available — the OS gives the illusion that enough memory exists.
- Instead of loading one whole big process, the OS loads parts of many processes into RAM. This raises the degree of multiprogramming and CPU utilization.
- Virtual address space = how the process sees its own memory (logical addresses, starting from 0).
- Physical memory = organized in fixed-size page frames.
- The MMU (Memory Management Unit) converts logical (virtual) addresses to physical addresses.

How it actually runs

- When a page is needed but RAM is full, the OS finds a page in RAM that is least used and moves it to disk to free space.
- This happens automatically, so the computer feels like it has unlimited RAM.
- Virtual memory can be implemented using: **Demand Paging** or **Demand Segmentation**.

In one line: Virtual Memory = disk pretends to be extra RAM, managed automatically by the OS.

3. Demand Paging

Demand Paging

Simple meaning: Pages of a process stay on disk and are loaded into RAM only when actually needed (on demand), not all at once.

- On a context switch, the OS does NOT preload pages. It just starts the new program and loads pages only as they are referenced.
- If a needed page is not in RAM, the CPU treats this as a Page Fault and gives control to the OS to fetch that page.

Page Fault

Simple meaning: A Page Fault happens when a program needs a page, but that page is not currently in RAM.

In one line: Page Fault = required page is missing from RAM.

Valid–Invalid Bit

- Every page table entry has a valid–invalid bit.
- v = page is in RAM (valid).
- i = page is NOT in RAM (invalid) — starts as i for all entries.
- If the MMU sees bit = i during address translation → Page Fault occurs.

Steps to Handle a Page Fault

1. CPU refers to a page → trap to OS (page fault).
2. OS checks: is this reference invalid (abort) or just not in memory?

3. OS finds a free frame in RAM.
4. OS swaps the needed page from disk into that frame.
5. Page table is updated; valid bit set to v.
6. The instruction that caused the fault is restarted.

Quick Revision

- Demand paging = load pages only when needed.
- Page Fault = required page not in RAM.
- Valid bit (v) = in RAM, Invalid bit (i) = not in RAM.
- 6 steps handle every page fault, ending with restarting the instruction.

4. Page Replacement

If RAM has no free frame when a page fault occurs, the OS must remove (replace) some existing page to make room. This is called Page Replacement.

Basic Page Replacement Steps

7. Find the needed page on disk.
8. Find a free frame. If none is free, choose a victim page using a replacement algorithm (write it back to disk first if it was changed/'dirty').
9. Bring the needed page into the free frame; update page/frame tables.
10. Restart the instruction that caused the fault.

Goal: choose a replacement algorithm that gives the minimum number of page faults.

5. Page Replacement Algorithms

All examples below use this reference string with 3 frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

FIFO (First-In-First-Out)

Main idea: Replace the page that has been in RAM the longest (oldest page), regardless of use.

- Simple to implement using a queue of pages.
- For the example reference string with 3 frames → 15 page faults.

Key point: FIFO can suffer from Belady's Anomaly — adding more frames can sometimes cause MORE page faults, not fewer (unexpected and undesirable).

Optimal Algorithm

Main idea: Replace the page that will NOT be used for the longest time in the future.

- Gives the lowest possible number of page faults — it is the theoretical best.
- Not practical: the OS cannot know the future references of a program.
- Used mainly to measure how good other algorithms are.

LRU (Least Recently Used)

Main idea: Replace the page that has NOT been used for the longest time in the past (uses history, not future).

- Approximates the Optimal algorithm using past behavior.
- Needs extra bookkeeping (e.g., counters or a stack) to track recent use.

Key point: LRU looks backward (past); Optimal looks forward (future, not possible in practice).

Quick Revision

Algorithm	Idea	Practical?
FIFO	Remove oldest page	Yes (simple, but weak)
Optimal	Remove page needed farthest in future	No (needs future knowledge)

Algorithm	Idea	Practical?
LRU	Remove page unused for longest time	Yes (good approximation)

Note: More frames usually mean fewer page faults, but FIFO can break this rule (Belady's Anomaly).

6. Thrashing

Thrashing

Simple meaning: Thrashing happens when the OS spends more time swapping pages between RAM and disk than actually running programs. Performance drops sharply.

In one line: Thrashing = too much page swapping, too little real work.

Causes of Thrashing

- High degree of multiprogramming: too many processes share limited frames, so each process gets very few frames → frequent page faults.
- Example: 400 free frames. With 100 processes → 4 frames each (fine). With 400 processes → 1 frame each (thrashing).
- Lack of frames: if a process gets too few frames, its pages keep swapping in and out.

CPU Utilization vs Degree of Multiprogramming

- As more processes are added, CPU utilization rises — up to an optimal point (λ).
- Beyond that point, adding more processes causes CPU utilization to fall sharply — this is thrashing.

Recovery from Thrashing

- Prevention: long-term scheduler should stop admitting new processes once the system nears the threshold.
- Cure: if already thrashing, the OS should suspend (swap out) some processes to free frames, then resume normal work.

Quick Revision

- Thrashing = excessive paging, low CPU work.
- Caused by too many processes or too few frames per process.
- Fixed by limiting multiprogramming or suspending processes.

7. Paging vs Segmentation

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments
Managed by	Operating System	Compiler
Speed	Faster	Slower
Closer to	Operating System	User / program structure
Fragmentation	Internal	External
Logical address	Page number + page offset	Segment number + segment offset
Table used	Page table	Segment table

Short difference: Paging splits memory into equal fixed blocks (simple, OS-controlled). Segmentation splits memory by logical program parts (variable size, closer to how a programmer thinks).

8. Segmented Paging

Simple meaning: Combines Segmentation and Paging. Main memory is divided into variable-size segments, and each segment is further divided into fixed-size pages.

Think of a library: Segments are big sections (Fiction, Science, History) — different sizes. Pages are equal-size shelves inside each section.

- Each segment has its own page table, so a program can have multiple page tables.
- Logical address (in theory) = Segment Number + Page Number + Page Offset.
- But the CPU only gives (Segment Number S, Offset d). We must split d ourselves into page number and page offset — that is the whole 'math' in this topic.

Advantages

- Reduces memory usage; no external fragmentation.
- Page table size is limited by segment size; simplifies allocation.

Disadvantages

- Still has internal fragmentation.
- More complex than plain paging; page tables must be stored contiguously.

The Two Tables Involved

- Segment Table (one row per segment): Limit = allowed size of the segment (used to check for errors); Page Table Base = memory address where this segment's own page table starts.
- Page Table (one per segment): each row stores which Frame in RAM holds that page.

Address Translation — Step by Step

11. Check validity: is Offset $d < \text{Segment Limit}$? If not → Error (invalid address).
12. Find Page Number: $p = d \div \text{Page Size}$ (integer division, drop the remainder).
13. Find Page Offset: $po = d \bmod \text{Page Size}$ (the remainder).
14. Look up page p in that segment's page table → get the Frame Number.
15. Compute Physical Address = (Frame Number $\times$ Page Size) + Page Offset.

In short: check limit → split d into (p, po) → find frame for p → combine frame with po.

Worked Example 1

Given: Segment = S0, Offset $d = 4$, Page Size = 2.

Segment Table → S0: Limit = 10, Page Table Base = 1000.

Page Table (for S0) → P0:F3, P1:F2, P2:F1, P3:F0.

Step 1: Is $4 < 10$? Yes → valid.

Step 2: Page Number $p = 4 \div 2 = 2$ → this is page P2.

Step 3: Page Offset $po = 4 \bmod 2 = 0$.

Step 4: P2 maps to Frame F1 → Frame Number = 1.

Step 5: Physical Address = $(1 \times 2) + 0 = 2$

Worked Example 2 (Practice, from PDF)

Given: Segment = S2, Offset $d = 6$, Page Size = 2.

Segment Table → S2: Limit = 12, Page Table Base = 1400.

Page Table (for S2) → P0:F3, P1:F2, P2:F1, P3:F0.

Step 1: Is $6 < 12$? Yes → valid.

Step 2: Page Number $p = 6 \div 2 = 3$ → this is page P3.

Step 3: Page Offset $po = 6 \bmod 2 = 0$.

Step 4: P3 maps to Frame F0 → Frame Number = 0.

Step 5: Physical Address = $(0 \times 2) + 0 = 0$

Quick Revision

- Segmented paging = segments (variable) split further into pages (fixed).
- Each segment has its own page table.
- 5-step recipe: check limit → $p = d \div \text{size}$ → $po = d \bmod \text{size}$ → find frame → Physical Address = (Frame $\times$ Page Size) + po.

Final Quick Revision

Important Terms

- Virtual Memory: disk used as extra RAM so programs feel unlimited memory.
- Demand Paging: pages loaded only when needed.
- Page Fault: required page not found in RAM.
- Page Replacement: removing a page from RAM to make space for a new one.
- Thrashing: system spends more time swapping pages than doing real work.
- MMU: hardware unit that converts logical address to physical address.

Important Formulas

- Page Number = $\text{Offset} \div \text{Page Size}$
- Page Offset = $\text{Offset} \bmod \text{Page Size}$
- Physical Address = $(\text{Frame Number} \times \text{Page Size}) + \text{Page Offset}$

Important Algorithms (one-line idea)

- FIFO: remove the oldest page in RAM.
- Optimal: remove the page needed farthest in the future (ideal, not practical).
- LRU: remove the page unused for the longest past time (practical, good approximation).

Important Comparison

Feature	Paging	Segmentation
Block size	Fixed	Variable
Fragmentation	Internal	External
Managed by	OS	Compiler

Most Important Concepts

- Virtual memory lets programs run even if they are bigger than physical RAM.
- Page faults are normal and are fixed by page replacement algorithms.
- Too little RAM per process → thrashing → system slows down badly.
- Segmented paging combines the benefits of both paging and segmentation.